

TỐI ƯU KHÔNG CÓ RÀNG BUỘC

THUẬT TOÁN

Hà Nội - 2024

ORLab, Trường Công nghệ thông tin Phenikaa
Đại học PHENIKAA

Phương pháp gradient

Cho $f : \mathbb{R}^n \rightarrow \mathbb{R}$ là hàm số khả vi liên tục.

Cho $f : \mathbb{R}^n \rightarrow \mathbb{R}$ là hàm số khả vi liên tục.

Xét bài toán tối ưu không có ràng buộc

$$\min\{f(x) : x \in \mathbb{R}^n\}.$$

Cho $f : \mathbb{R}^n \rightarrow \mathbb{R}$ là hàm số khả vi liên tục.

Xét bài toán tối ưu không có ràng buộc

$$\min\{f(x) : x \in \mathbb{R}^n\}.$$

- Giải phương trình $\nabla f(x) = 0$ tìm điểm dừng

Cho $f : \mathbb{R}^n \rightarrow \mathbb{R}$ là hàm số khả vi liên tục.

Xét bài toán tối ưu không có ràng buộc

$$\min\{f(x) : x \in \mathbb{R}^n\}.$$

- Giải phương trình $\nabla f(x) = 0$ tìm điểm dừng
- Nếu f là hàm lồi thì điểm dừng là nghiệm của bài toán.

Thuật toán lặp đi tìm điểm dừng

$$x_{k+1} = x_k + t_k d_k$$

- ▶ d_k : hướng (direction)
- ▶ t_k : bước lặp (stepsize)

Definition

Cho $f : \mathbb{R}^n \rightarrow \mathbb{R}$ là hàm số khả vi liên tục. Một vectơ $d \in \mathbb{R}^n$ được gọi là một **hướng giảm** (*descent direction*) của f tại x nếu đạo hàm theo hướng $f'(x; d) < 0$, tức là

$$f'(x; d) = \nabla f(x)^T d < 0.$$

Lemma

Cho f là hàm số khả vi liên tục trên $\mathbb{R}^n$ và $x \in \mathbb{R}^n$. Giả sử rằng d là một hướng giảm của f tại x . Khi đó, tồn tại $\epsilon > 0$ sao cho

$$f(x + td) < f(x)$$

với mọi $t \in (0, \epsilon]$.

- 👉 A general descent directions method

👉 A general descent directions method

Khởi tạo: Xuất phát từ $x_0 \in \text{dom}f$ bất kỳ

👉 A general descent directions method

Khởi tạo: Xuất phát từ $x_0 \in \text{dom}f$ bất kỳ

Ở mỗi bước: $k = 0, 1, 2, \dots$

👉 A general descent directions method

Khởi tạo: Xuất phát từ $x_0 \in \text{dom}f$ bất kỳ

Ở mỗi bước: $k = 0, 1, 2, \dots$

(1) Hướng giảm (descent direction) d_k và bước lặp (stepsize) t_k được chọn sao cho

$$f(x_k + t_k d_k) < f(x_k).$$

👉 A general descent directions method

Khởi tạo: Xuất phát từ $x_0 \in \text{dom}f$ bất kỳ

Ở mỗi bước: $k = 0, 1, 2, \dots$

(1) Hướng giảm (descent direction) d_k và bước lặp (stepsize) t_k được chọn sao cho

$$f(x_k + t_k d_k) < f(x_k).$$

(2) $x_{k+1} := x_k + t_k d_k$.

👉 A general descent directions method

Khởi tạo: Xuất phát từ $x_0 \in \text{dom}f$ bất kỳ

Ở mỗi bước: $k = 0, 1, 2, \dots$

(1) Hướng giảm (descent direction) d_k và bước lặp (stepsize) t_k được chọn sao cho

$$f(x_k + t_k d_k) < f(x_k).$$

(2) $x_{k+1} := x_k + t_k d_k$.

(3) Nếu tiêu chuẩn dừng thỏa mãn thì STOP và x_k là **output**

????????????

- ▶ Bước lặp (stepsize) chọn như thế nào?

????????????

- ▶ Bước lặp (stepsize) chọn như thế nào?
- ▶ Tiêu chuẩn nào để dừng thuật toán?

Hướng giảm: ngược hướng gradient

$d_k = -\nabla f(x_k)$ là một hướng giảm tại x_k nếu $\nabla f(x_k) \neq 0$.

Hướng giảm: ngược hướng gradient

$d_k = -\nabla f(x_k)$ là một hướng giảm tại x_k nếu $\nabla f(x_k) \neq 0$. Thật vậy

$$f'(x_k; -\nabla f(x_k)) = -\nabla f(x_k)^T \nabla f(x_k) = -\|\nabla f(x_k)\|^2 < 0.$$

Gradient descent method

Điểm xuất phát $x_0 \in \text{dom} f$

- ▶ Chọn **bước lặp** (*stepsize*) t_k là **hằng số**,

Gradient descent method

Điểm xuất phát $x_0 \in \text{dom} f$

- ▶ Chọn **bước lặp** (*stepsize*) t_k là **hằng số**, **line search**,

Gradient descent method

Điểm xuất phát $x_0 \in \text{dom} f$

- ▶ Chọn **bước lặp** (stepsize) t_k là **hằng số**, **line search**, hay **backtracking (line search)**
- ▶ Thực hiện $x_{k+1} := x_k - t_k \nabla f(x_k)$ cho đến khi tiêu chuẩn dừng thỏa mãn.

Bước lặp	x_k	$\ \nabla f(x_k)\ $
k=0	...	...
k=1	...	...
k=2	...	...
k=3	...	...
$\vdots$	$\vdots$	$\vdots$
...	...	...

- ▶ Tiêu chuẩn dừng thuật toán:

$$\|\nabla f(x_{k+1})\|_2 < \epsilon$$

- ▶ Tiêu chuẩn dừng thuật toán:

$$\|\nabla f(x_{k+1})\|_2 < \epsilon$$

- ϵ là một số dương đủ bé được chọn trước (input)

- Tiêu chuẩn dừng thuật toán:

$$\|\nabla f(x_{k+1})\|_2 < \epsilon$$

- ϵ là một số dương đủ bé được chọn trước (input)
- ϵ được gọi là tham số dung sai (*tolerance parameter*) (Thông thường, cho $\epsilon = 10^{-6}$ hoặc $\epsilon = 10^{-5}$)

👉 **Nhắc lại:** Chuẩn (Euclide) của một vectơ $z = (z_1, z_2, \dots, z_n)$ trong $\mathbb{R}^n$ được ký hiệu là $\|z\|_2$, được xác định bởi công thức

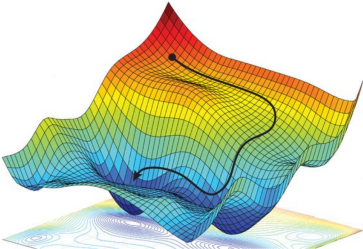
$$\|z\|_2 = \sqrt{z_1^2 + z_2^2 + \dots + z_n^2}.$$

Thư đã gửi - huyen.duan... x | gradient descent - Goog... x | gradient descent - Tìm t... x | [Gradient Descent] - Ph... x | medium.com x | + -

Không bảo mật tutorials.aiclub.cs.uit.edu.vn/index.php/2020/06/07/machine-learning-gradient-descent-la-gi-phan-1-5/ ☆

[Gradient Descent] – Phần 1: “Gradient Descent là gì?”

June 7, 2020 Khoa Nguyen Van



Hình 1: <https://towardsdatascience.com/coding-deep-learning-for-beginners-linear-regression-gradient->

RECENT POSTS

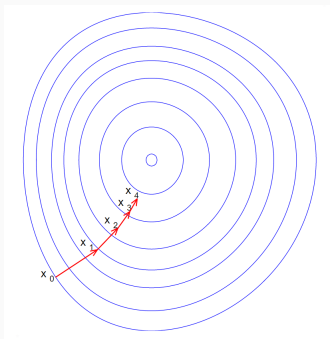
- [AI-Challenge 2021] Nhận dạng chữ tiếng Việt trong ảnh ngoại cảnh
- [Machine Learning] Shopping with Association rule
- [Machine Learning] Bias và Variance

META

- Log in
- Entries feed
- Comments feed
- WordPress.org

Type here to search

10:34 AM 1/12/2024



Hình 1: Gradient Method

Chọn bước lặp (stepsize)

- ▶ Bước lặp **hằng** $t_k = t$ với mọi k

Chọn bước lặp (stepsize)

- ▶ Bước lặp **hằng** $t_k = t$ với mọi k
- ▶ Exact line search

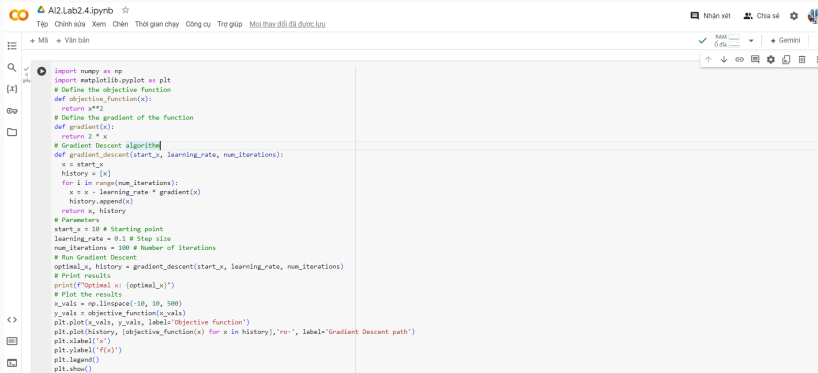
Chọn bước lặp (stepsize)

- ▶ Bước lặp **hằng** $t_k = t$ với mọi k
- ▶ Exact line search
- ▶ Backtracking

t_k là cực tiểu dọc theo tia $\{x_k + td_k \mid t_k \geq 0\}$:

$$t_k = \operatorname{argmin}_{t \geq 0} f(x_k + td_k)$$

Ví dụ minh họa (Lab 2.4, AI2)



```
import numpy as np
import matplotlib.pyplot as plt

# Define the objective function
def objective_function(x):
    return x**2

# Define the gradient of the function
def gradient(x):
    return 2 * x

# Gradient Descent algorithm
def gradient_descent(start_x, learning_rate, num_iterations):
    x = start_x
    history = [x]
    for i in range(num_iterations):
        x = x - learning_rate * gradient(x)
        history.append(x)
    return x, history

# Parameters
start_x = 10 # Starting point
learning_rate = 0.1 # Step size
num_iterations = 100 # Number of iterations

# Run Gradient Descent
optimal_x, history = gradient_descent(start_x, learning_rate, num_iterations)

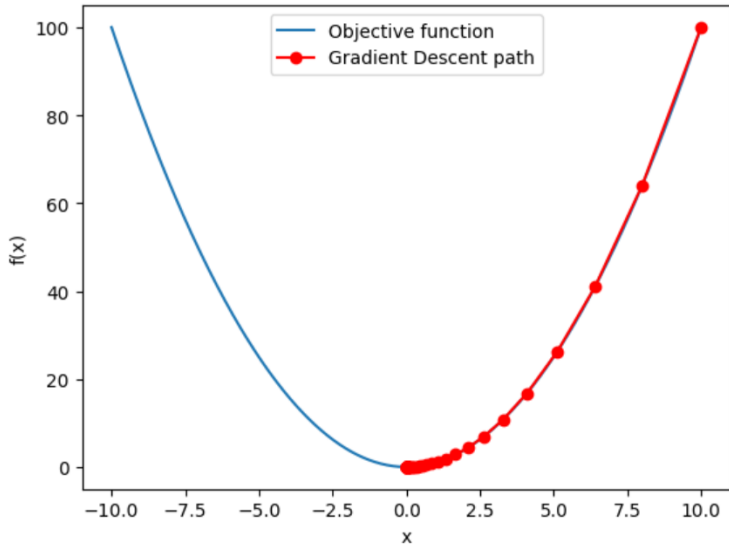
# Print results
print(f"Optimal x: {optimal_x}")

# Plot the results
x_vals = np.linspace(-10, 10, 500)
y_vals = objective_function(x_vals)

plt.plot(x_vals, y_vals, label='Objective function')
plt.plot(history, [objective_function(x) for x in history], 'ro-', label='Gradient Descent path')
plt.xlabel('x')
plt.ylabel('f(x)')
plt.legend()
plt.show()
```

Ví dụ minh họa (Lab 2.4, AI2)

Optimal x : $2.0370359763344878e-09$



Backtracking (line search)

Hầu hết các phương pháp line search dùng trong thực hành là không chính xác (inexact). Bước lặp được chọn sao cho cực tiểu xấp xỉ hàm mục tiêu f theo tia $\{x_k + td_k \mid t_k \geq 0\}$ hoặc là giảm f vừa đủ.

Backtracking (line search)

Hầu hết các phương pháp line search dùng trong thực hành là không chính xác (inexact). Bước lặp được chọn sao cho cực tiểu xấp xỉ hàm mục tiêu f theo tia $\{x_k + td_k \mid t_k \geq 0\}$ hoặc là giảm f vừa đủ.

👉 Backtracking là một phương pháp “inexact line search” đơn giản và hiệu quả

Backtracking line search

- ▶ Cho một hướng giảm d của f tại $x \in \text{dom} f$, $\alpha \in (0, 0.5)$, $\beta(0, 1)$, $t = 1$.
- ▶ Chừng nào $f(x + td) > f(x) + \alpha t \nabla f(x)^T d$, $t := \beta t$

Phương pháp gọi là “backtracking” bởi vì nó bắt đầu với stepsize đơn vị và rồi nó giảm dần bằng việc lấy tích với β cho đến khi điều kiện dừng $f(x + td) \leq f(x) + \alpha t \nabla f(x)^T d$ thỏa mãn.

👉 **Ví dụ:** Xây dựng dãy lặp theo phương pháp gradient cho bài toán

$$\{\min f(x, y) = x^2 + 2y^2 : (x, y) \in \mathbb{R}^2\}$$

Điểm bắt đầu: $(x_0, y_0) = (2, 1)$

Stepsize: $t = 0.1$

Sự hội tụ và tốc độ hội tụ của thuật toán gradient

Một số sự lựa chọn của search direction d ¹

- ▶ Hướng giảm nhanh nhất (steepest descent, Cauchy direction)

$$d_k = \frac{\nabla f(x_k)}{\|\nabla f(x_k)\|}$$

¹<https://sites.math.washington.edu/burke/crs/516/notes/backtracking.pdf>

Một số sự lựa chọn của search direction d ¹

- ▶ Hướng giảm nhanh nhất (steepest descent, Cauchy direction)

$$d_k = \frac{\nabla f(x_k)}{\|\nabla f(x_k)\|}$$

- ▶ Hướng giảm Newton

$$d_k = \frac{\nabla f(x_k)}{\|\nabla^2 f(x_k)\|}$$

¹<https://sites.math.washington.edu/burke/crs/516/notes/backtracking.pdf>

Một số sự lựa chọn của search direction d ¹

- ▶ Hướng giảm nhanh nhất (steepest descent, Cauchy direction)

$$d_k = \frac{\nabla f(x_k)}{\|\nabla f(x_k)\|}$$

- ▶ Hướng giảm Newton

$$d_k = \frac{\nabla f(x_k)}{\|\nabla^2 f(x_k)\|}$$

- ▶ Hướng giảm tựa Newton

¹<https://sites.math.washington.edu/~burke/crs/516/notes/backtracking.pdf>

Newton method

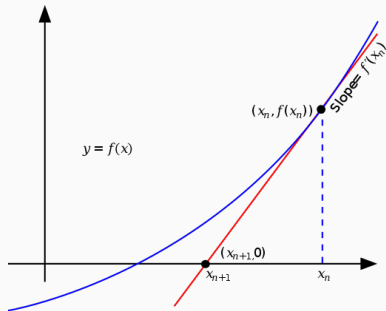
Điểm xuất phát $x_0 \in \text{dom} f$

- Thực hiện $x_{k+1} := x_k - \frac{\nabla f(x_k)}{\|\nabla^2 f(x_k)\|}$ cho đến khi tiêu chuẩn dừng thỏa mãn.

Newton method

Điểm xuất phát $x_0 \in \text{dom} f$

- Thực hiện $x_{k+1} := x_k - \frac{\nabla f(x_k)}{\|\nabla^2 f(x_k)\|}$ cho đến khi tiêu chuẩn dừng thỏa mãn.



Hình 2: Newton Method

Example

Ví dụ: Tìm cực tiểu hàm số $f(x) = \frac{x^2}{2} - \sin(x)$ bằng phương pháp Newton

Example

Ví dụ: Tìm cực tiểu hàm số $f(x) = \frac{x^2}{2} - \sin(x)$ bằng phương pháp Newton

- ▶ Chứng minh hàm mục tiêu là lồi
- ▶ Thực hành trên Python, mô tả dãy lặp trên đồ thị.

Exercise

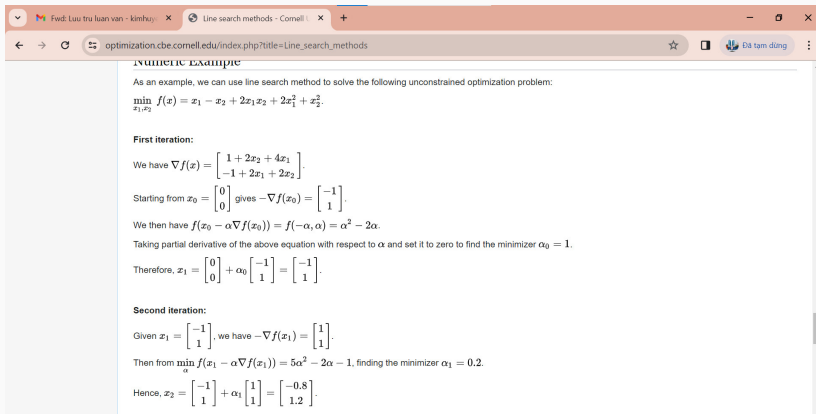
Ví dụ: Thực hành giải bài toán sau bằng các phương pháp: gradient, Newton, và line search. Tìm cực tiểu hàm số

$$f(x_1, x_2) = x_1 - x_2 + 2x_1x_2 + 2x_1^2 + x_2^2$$

Exercise

Ví dụ: Thực hành giải bài toán sau bằng các phương pháp: gradient, Newton, và line search. Tìm cực tiểu hàm số

$$f(x_1, x_2) = x_1 - x_2 + 2x_1x_2 + 2x_1^2 + x_2^2$$



The screenshot shows a web browser window with the address bar displaying 'optimization.cbe.cornell.edu/index.php?title=Line_search_methods'. The page content includes a section titled 'NUMERIC EXAMPLE' and a paragraph stating: 'As an example, we can use line search method to solve the following unconstrained optimization problem: $\min_{x_1, x_2} f(x) = x_1 - x_2 + 2x_1x_2 + 2x_1^2 + x_2^2$.'

First iteration:

We have $\nabla f(x) = \begin{bmatrix} 1 + 2x_2 + 4x_1 \\ -1 + 2x_1 + 2x_2 \end{bmatrix}$.

Starting from $x_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$ gives $-\nabla f(x_0) = \begin{bmatrix} -1 \\ 1 \end{bmatrix}$.

We then have $f(x_0 - \alpha \nabla f(x_0)) = f(-\alpha, \alpha) = \alpha^2 - 2\alpha$.

Taking partial derivative of the above equation with respect to α and set it to zero to find the minimizer $\alpha_0 = 1$.

Therefore, $x_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \alpha_0 \begin{bmatrix} -1 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \end{bmatrix}$.

Second iteration:

Given $x_1 = \begin{bmatrix} -1 \\ 1 \end{bmatrix}$, we have $-\nabla f(x_1) = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$.

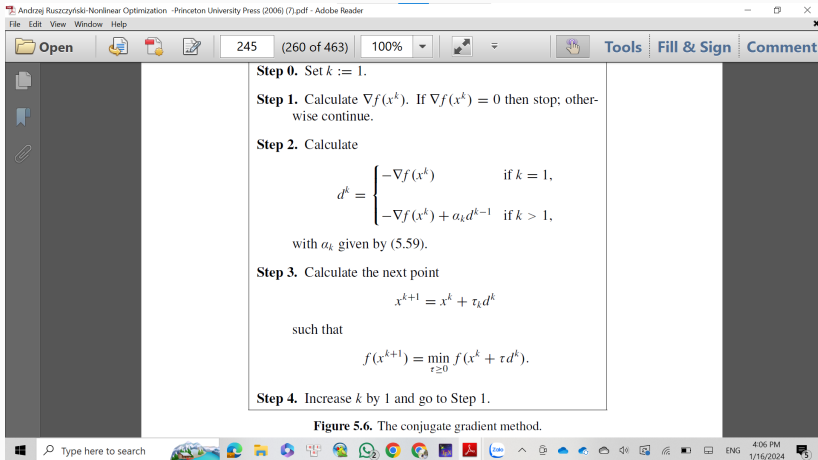
Then from $\min_{\alpha} f(x_1 - \alpha \nabla f(x_1)) = 5\alpha^2 - 2\alpha - 1$, finding the minimizer $\alpha_1 = 0.2$.

Hence, $x_2 = \begin{bmatrix} -1 \\ 1 \end{bmatrix} + \alpha_1 \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} -0.8 \\ 1.2 \end{bmatrix}$.

- ▶ Chứng minh hàm mục tiêu là lồi
- ▶ Thực hành trên Python, mô tả các dãy lặp sinh ra bởi các phương pháp lặp (cùng điểm xuất phát) trên đồ thị.

Phân tích sự hội tụ và tốc độ hội tụ

Phương pháp hướng gradient liên hợp (conjugate-gradient method)



$$\alpha_k = \frac{\langle \nabla f(x_k), \nabla f(x_k) - \nabla f(x_{k-1}) \rangle}{\|\nabla f(x_{k-1})\|^2}$$

²Andrzej P. Ruszczyński, Nonlinear optimization, Princeton University Press, 2006

Xét bài toán cực tiểu hàm toàn phương ³

$$f(x) = \frac{1}{2}\langle x, Qx \rangle + \langle c, x \rangle$$

³Andrzej P. Ruszczyński, Nonlinear optimization, Princeton University Press, 2006

Xét bài toán cực tiểu hàm toàn phương³

$$f(x) = \frac{1}{2}\langle x, Qx \rangle + \langle c, x \rangle$$

👉 Step size được tính hiển như sau

$$\tau_k = -\frac{\langle \nabla f(x_k), d_k \rangle}{\langle d_k, Qd_k \rangle} = \frac{\|\nabla f(x_k)\|^2}{\langle d_k, Qd_k \rangle}$$

³Andrzej P. Ruszczyński, Nonlinear optimization, Princeton University Press, 2006

Xét bài toán cực tiểu hàm toàn phương³

$$f(x) = \frac{1}{2}\langle x, Qx \rangle + \langle c, x \rangle$$

👉 Step size được tính hiển như sau

$$\tau_k = -\frac{\langle \nabla f(x_k), d_k \rangle}{\langle d_k, Qd_k \rangle} = \frac{\|\nabla f(x_k)\|^2}{\langle d_k, Qd_k \rangle}$$

Chú ý là $d_k = -\nabla f(x_k)$

³Andrzej P. Ruszczyński, Nonlinear optimization, Princeton University Press, 2006

